

EXPERIMENT NO. 3

Name of Student	<u>Anushka Shahane</u>
Class Roll No	D15A - 54
D.O.P.	20/2/25
D.O.S.	27/2/25
Sign and Grade	

AIM : To develop a basic Flask application with multiple routes and demonstrate the handling of GET and POST requests.

PROBLEM STATEMENT :

Design a Flask web application with the following features:

1. A homepage (/) that provides a welcome message and a link to a contact form.
 - a. Create routes for the homepage (/), contact form (/contact), and thank-you page (/thank_you).
2. A contact page (/contact) where users can fill out a form with their name and email.
3. Handle the form submission using the POST method and display the submitted data on a thank-you page (/thank_you).
 - a. On the contact page, create a form to accept user details (name and email).
 - b. Use the POST method to handle form submission and pass data to the thank-you page
4. Demonstrate the use of GET requests by showing a dynamic welcome message on the homepage when the user accesses it with a query parameter, e.g., /welcome?name=<user_name>.
 - a. On the homepage (/), use a query parameter (name) to display a personalized welcome message.

Theory:

1. Core Features of Flask

Flask is a lightweight Python web framework that is widely appreciated for its simplicity and flexibility. Here are the core features of Flask:

- **Minimalistic Design:** Flask is often called a "micro-framework" because it comes with the basic essentials for building web apps. It doesn't force you into a specific structure or tools but gives you the freedom to choose the libraries and tools you need.
- **Routing:** Flask allows you to define routes that map URLs to Python functions. This is how Flask responds to HTTP requests, making routing one of its key features. With Flask's routing, you can easily handle different HTTP methods such as GET, POST, PUT, and DELETE.
- **Templating Engine:** Flask uses **Jinja2** as its default templating engine. Jinja2 allows you to insert dynamic content into HTML templates, making it easier to create web pages that adapt to different inputs (e.g., displaying user data).
- **Request and Response Handling:** Flask has built-in support for handling requests and responses. It allows you to access data submitted via forms or query parameters, manipulate it, and send back responses in various formats (HTML, JSON, etc.).
- **Session Management:** Flask supports managing sessions, which allows you to store data between different requests. This is crucial for maintaining user logins or storing temporary user data.
- **Extensibility:** Flask provides the ability to add various extensions to handle things like database connections (e.g., **Flask-SQLAlchemy**), form validation (e.g., **Flask-WTF**), and user authentication (e.g., **Flask-Login**).
- **Development Server & Debugger:** Flask comes with a built-in development server and interactive debugger. This allows you to quickly iterate on your application during development, with real-time error messages and live code reloading.
- **RESTful Support:** Flask is great for building APIs, especially **RESTful APIs**. Flask's lightweight nature makes it ideal for creating backend services that respond with JSON, and it provides tools for defining routes and handling HTTP methods easily.

2. Why Do We Use Flask(__name__) in Flask?

In a Flask application, you initialize the app using:

```
app = Flask(__name__)
```

- **__name__ represents the current Python module or script:** It is a built-in Python variable that gets its value depending on the context. If the Python file is being run directly, __name__ is set to '__main__', and if it is being imported as a module, __name__ gets set to the name of the module.
- **Helps Flask locate resources:** Flask uses the __name__ value to determine where to look for resources, such as templates, static files, and the location of the app itself. It helps Flask understand where the application resides in the file structure.
- **Development Environment Setup:** When Flask(__name__) is used, Flask is able to set up a proper application context for debugging and development purposes. It makes it possible to activate things like the Flask development server and debugging features.

By passing __name__, Flask knows which resources to load and ensures everything works properly in both development and production environments.

3. What is Template (Template Inheritance) in Flask?

In Flask, **templates** are HTML files that can contain placeholders for dynamic content. Flask uses **Jinja2** as the templating engine to handle these templates.

- **Templates:** These are files where dynamic content can be inserted into static HTML. For example, if you need to display user names, you can use placeholders in the HTML template that will be filled with data from the server when the page is rendered.

Example of a simple template (hello.html):

```
<!DOCTYPE html>

<html>

  <head>

    <title>Hello, {{ name }}!</title>

  </head>

  <body>

    <h1>Hello, {{ name }}!</h1>

  </body>

</html>
```

Here, `{{ name }}` is a placeholder that Flask will replace with the actual name value when rendering the template.

- **Template Inheritance:** Flask allows you to create a **base template** that can be extended by other templates. This promotes code reusability and avoids redundancy in HTML structures.

For example, a **base template** (base.html) could define the common structure (header, footer) for your pages:

```
<!DOCTYPE html>

<html>

  <head>

    <title>{% block title %}My Website{% endblock %}</title>

  </head>

  <body>

    <header>

      <h1>My Website</h1>

    </header>
```

```
<div>

    {% block content %} {% endblock %}

</div>


<footer>

    <p>&copy; 2025</p>

</footer>

</body>

</html>
```

Then, other templates can extend this base template and define their own content for the content block:

```
{% extends 'base.html' %}

{% block title %}Home Page{% endblock %}


{% block content %}

    <h2>Welcome to the Home Page</h2>

    <p>This is the content of the home page.</p>

{% endblock %}
```

This way, only the content part changes, and the common structure (e.g., header, footer) is inherited from the base template.

4. What Methods of HTTP Are Implemented in Flask?

Flask supports all the standard HTTP methods used for web communication. The following are the most commonly used HTTP methods in Flask:

- **GET**: Used to retrieve data from the server. It's the default method when accessing web pages. In Flask, it's used to display content or data.
- **POST**: Used to send data to the server, usually when submitting a form or creating a new resource on the server.
- **PUT**: Used to update an existing resource on the server.
- **DELETE**: Used to delete a resource on the server.
- **PATCH**: Used to partially update a resource on the server, often with fewer data than PUT.
- **OPTIONS**: Used to describe the communication options for the target resource, commonly used in cross-origin resource sharing (CORS).

In Flask, you define which HTTP methods a route should handle using the `methods` parameter in the route decorator:

```
@app.route('/submit', methods=['POST'])  
  
def submit():  
  
    return 'Form Submitted'
```

5. Difference Between Flask and Django Framework

Feature	Flask	Django
Philosophy	Micro-framework with minimal overhead. Focus on flexibility and simplicity.	Full-stack framework with lots of built-in features to accelerate development.
Features	Comes with essential features like routing, templating, and basic session handling. Additional features can be added via extensions.	Comes with built-in tools for database integration (ORM), admin interface, user authentication, and more.
Learning Curve	Easier to learn, particularly for small projects. Ideal for beginners.	Steeper learning curve due to its "batteries-included" nature.
Extensibility	Highly extensible. You can add only what you need, and structure the app how you want.	Less flexible due to its "convention-over-configuration" approach. You follow the predefined structure.
Project Size	Best suited for small to medium projects or microservices.	Better for large applications, where you need built-in features and scalability.
Admin Interface	No built-in admin interface. You can add one with extensions like Flask-Admin.	Comes with a built-in admin interface that allows easy management of the database.
Database Integration	No built-in ORM, but you can use libraries like SQLAlchemy for database management.	Comes with its own powerful ORM (Object-Relational Mapping) to interact with databases.

Flexibility	Highly flexible and customizable. No forced structure.	Tightly structured. It imposes a specific project layout and coding practices.
--------------------	---	--

Routing

1. Routing in Flask

Routing in Flask refers to how URLs are mapped to functions. In other words, when a user visits a URL in a web browser, Flask needs to know what Python code to execute to generate a response. Routing allows us to define these URL-to-function mappings.

In Flask, you define a route by using the `@app.route()` decorator. This decorator takes a URL pattern and maps it to a specific function. The function is called a view function and is responsible for handling the request and returning a response.

Example of Routing:

```
from flask import Flask

app = Flask(__name__)

@app.route('/')
def home():
    return 'Welcome to the Home Page!'

@app.route('/about')
def about():
    return 'This is the About Page'

if __name__ == '__main__':
    app.run()
```

In the example above:

- `/` is the root URL of the site. When the user visits the root URL, the `home()` function is triggered, and a "Welcome to the Home Page!" message is shown.

- /about is another URL that triggers the about() function, displaying the "This is the About Page" message.

2. URL Building in Flask

URL building refers to dynamically generating URLs for your routes, which helps in linking to different parts of your Flask application. Flask provides a function called `url_for()` to help build URLs programmatically based on the function name of the route.

Using `url_for()` is especially useful when your URLs may change (e.g., during refactoring) because it allows Flask to automatically generate the correct URL for a given route function, avoiding hardcoding URLs in the templates.

Example of URL Building:

```
from flask import Flask, url_for

app = Flask(__name__)

@app.route('/')
def home():
    return 'Home Page'

@app.route('/profile')
def profile():
    return 'Profile Page'

@app.route('/go_to_profile')
def go_to_profile():
    profile_url = url_for('profile') # This will dynamically generate '/profile'
    return f'Click here to go to your profile: <a href="{profile_url}">Go to Profile</a>'

if __name__ == '__main__':
    app.run()
```

Why use URL Building?

- Avoid hardcoding URLs throughout the app.
- It's more flexible and less error-prone.

- Makes the app easier to maintain because URLs are dynamically generated.

3. GET Request in Flask

A GET request is the most common type of HTTP request. When a user visits a webpage, the browser sends a GET request to the server to retrieve resources such as HTML, images, or other content. GET requests are used to retrieve data from the server.

- Characteristics:
 - GET requests do not modify any server-side data.
 - They are typically used for retrieving resources.
 - GET requests can pass data to the server through query parameters in the URL (e.g., /search?q=flask).

Handling GET Requests in Flask: By default, Flask routes respond to GET requests. You don't have to explicitly mention the method unless you want to restrict the route to only handle specific methods.

Example of GET Request:

```
@app.route('/search', methods=['GET'])
```

```
def search():
```

```
    query = request.args.get('q') # Extracts the query parameter 'q' from the URL
```

```
    return f'You searched for: {query}'
```

Here, when the URL /search?q=flask is visited:

- Flask receives the GET request, and the query parameter q=flask is extracted using request.args.get('q').
- The function returns a response, "You searched for: flask".

GET requests are idempotent, meaning making the same GET request multiple times will always return the same result, and they do not cause side effects on the server.

4. POST Request in Flask

A POST request is used when data is sent to the server to be processed, often in the form of form submissions. POST requests are used to create or modify data on the server. Unlike GET requests, POST requests send data in the body of the request rather than the URL.

- Characteristics:
 - POST requests send data to the server.

- They are used for actions like creating new records, submitting forms, or uploading files. The data sent in a POST request is not visible in the URL (unlike GET requests).
- POST requests can be more secure than GET requests, especially when transmitting sensitive data.

Handling POST Requests in Flask: To handle POST requests in Flask, you specify the `methods=['POST']` argument in the `route()` decorator.

Example of POST Request:

```
from flask import Flask, request
```

```
app = Flask(__name__)
```

```
@app.route('/submit', methods=['POST'])
```

```
def submit():
```

```
    username = request.form.get('username') # Get the 'username' form data
```

```
    password = request.form.get('password') # Get the 'password' form data
```

```
    return f'Username: {username}, Password: {password}'
```

```
if __name__ == '__main__':
```

```
    app.run()
```

In this example:

- A form that sends a POST request with fields like username and password is expected at `/submit`.
- The data from the form is extracted using `request.form.get('field_name')`.
- The server processes the data (e.g., saving to a database) and returns a response.

How POST Request Works:

- The browser sends the data in the body of the request when submitting a form.
- For example, if a user fills in a form with their username and password and clicks submit, the form data will be sent as a POST request to the server.

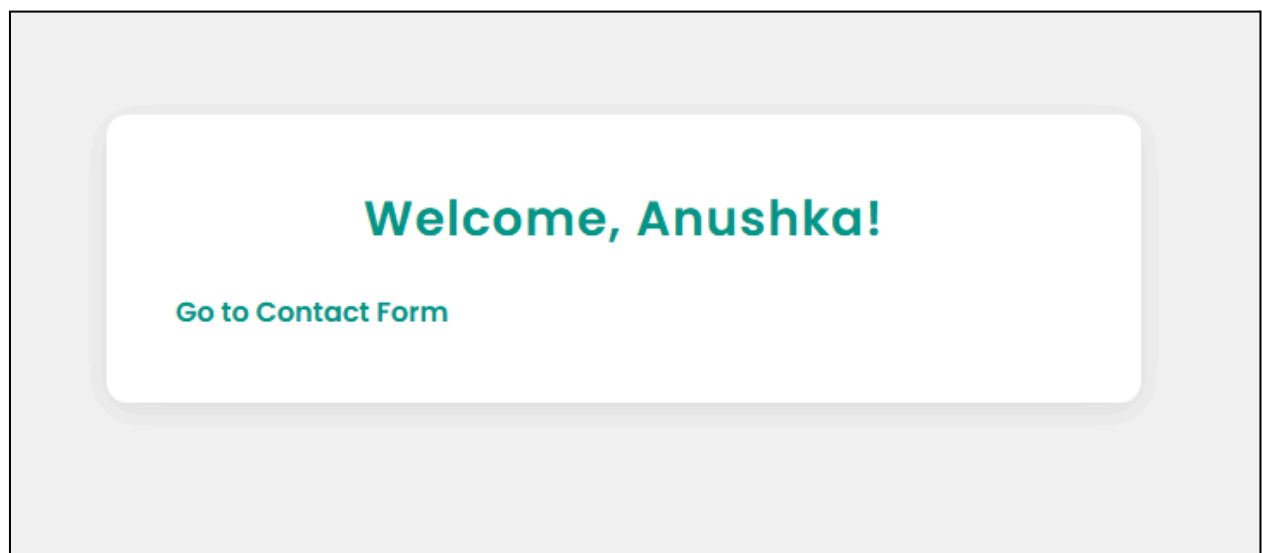
Example of HTML form sending POST request:

```
<form action="/submit" method="POST">  
  
  <input type="text" name="username" placeholder="Enter Username" required>  
  
  <input type="password" name="password" placeholder="Enter Password" required>  
  
  <button type="submit">Submit</button>  
  
</form>
```

This form sends a POST request to the /submit route when the user submits it, and the server can process the data accordingly.

OUTPUT

<http://127.0.0.1:5000/>



<http://127.0.0.1:5000/contact>

Contact Us

Name

Email

Submit

http://127.0.0.1:5000/thank_you?name=Anushka&email=anushka3204@gmail.com

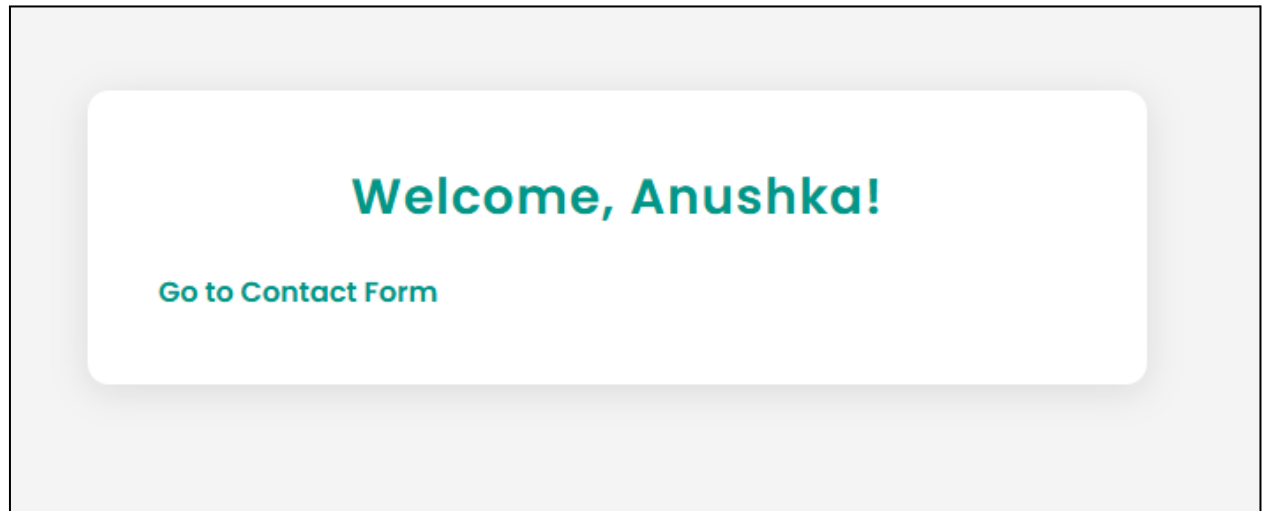
127.0.0.1:5000/thank_you?name=Anushka&email=anu@gmail.com

Thank You, Anushka!

Your contact information has been received. We will get back to you soon at anu@gmail.com.

[Back to Homepage](#)

<http://127.0.0.1:5000/?name=Anushka>



Github link : https://github.com/Anushka3204/Exp_3_WebX_flask

Conclusion : This experiment demonstrated the creation of a basic Flask web application using multiple routes and handling both GET and POST requests. It helped in understanding form submissions, dynamic URL handling using query parameters, and template rendering. Overall, it provided hands-on experience with the core functionalities of Flask essential for web development.